

Making a Library

Since the `rand` and `bubble_sort` functions in the previous program are useful, you will probably want to reuse them in other programs you write. You can put them into a utility library to make their reuse easier.

Every library consists of two parts: a header file and the actual code file. The header file, normally denoted by a `.h` suffix, contains information about the library that programs using it need to know. In general, the header file contains constants and types, along with prototypes for functions available in the library. Enter the following header file and save it to a file named **util.h**.

```
/* util.h */
extern int rand();
extern void bubble_sort(int, int []);
```

These two lines are function prototypes. The word "extern" in C represents functions that will be linked in later. If you are using an old-style compiler, remove the parameters from the parameter list of **bubble_sort**.

Enter the following code into a file named **util.c**.

```
/* util.c */
#include "util.h"

int rand_seed=10;

/* from K&R
- produces a random number between 0 and 32767.*/
int rand()
{
    rand_seed = rand_seed * 1103515245 +12345;
    return (unsigned int)(rand_seed / 65536) % 32768;
}

void bubble_sort(int m,int a[])
{
    int x,y,t;
    for (x=0; x < m-1; x++)
        for (y=0; y < m-x-1; y++)
            if (a[y] > a[y+1])
            {
                t=a[y];
                a[y]=a[y+1];
                a[y+1]=t;
            }
}
```

Note that the file includes its own header file (**util.h**) and that it uses quotes instead of the symbols `<` and `>`, which are used only for system libraries. As you can see, this looks like normal C code. Note that the variable **rand_seed**, because it is not in the header file, cannot be seen or modified by a program using this library. This is called information hiding. Adding the word **static** in front of **int** enforces the hiding completely.

Enter the following main program in a file named **main.c**.

```
#include <stdio.h>
#include "util.h"

#define MAX 10
```

```
int a[MAX];

void main()
{
    int i,t,x,y;
    /* fill array */
    for (i=0; i < MAX; i++)
    {
        a[i]=rand();
        printf("%d\n",a[i]);
    }

    bubble_sort(MAX,a);

    /* print sorted array */
    printf("-----\n");
    for (i=0; i < MAX; i++)
        printf("%d\n",a[i]);
}
```

This code includes the utility library. The main benefit of using a library is that the code in the main program is much shorter.

Compiling and Running with a Library

To compile the library, type the following at the command line (assuming you are using UNIX) (replace gcc with cc if your system uses cc):

```
gcc -c -g util.c
```

The **-c** causes the compiler to produce an object file for the library. The object file contains the library's machine code. It cannot be executed until it is linked to a program file that contains a main function. The machine code resides in a separate file named **util.o**.

To compile the main program, type the following:

```
gcc -c -g main.c
```

This line creates a file named **main.o** that contains the machine code for the main program. To create the final executable that contains the machine code for the entire program, link the two object files by typing the following:

```
gcc -o main main.o util.o
```

This links **main.o** and **util.o** to form an executable named **main**. To run it, type **main**.

Makefiles make working with libraries a bit easier. You'll find out about makefiles on the next page.